

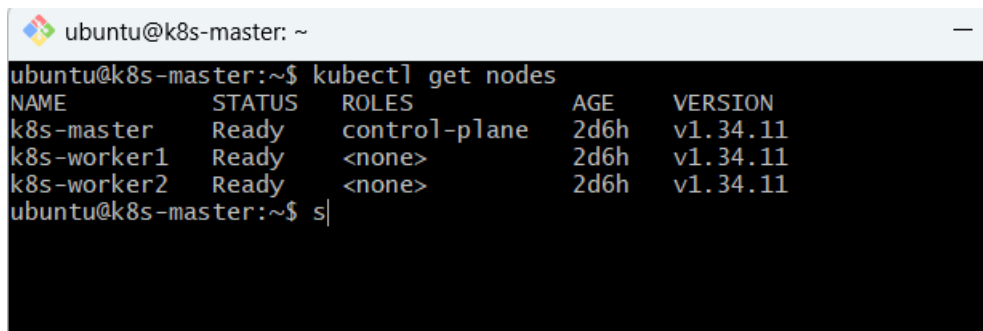
KUBERNETES (K8S) TASK - 04

1. Deploy an application using a Deployment with 3 replicas and a rolling update strategy.

Prerequisites

Before starting, verify:

kubectl get nodes

A terminal window titled 'ubuntu@k8s-master: ~' showing the command 'kubectl get nodes' and its output. The output is a table with 5 columns: NAME, STATUS, ROLES, AGE, and VERSION. It lists three nodes: 'k8s-master' (Ready, control-plane, 2d6h, v1.34.11), 'k8s-worker1' (Ready, <none>, 2d6h, v1.34.11), and 'k8s-worker2' (Ready, <none>, 2d6h, v1.34.11). The prompt 'ubuntu@k8s-master:~\$' is followed by 's|' on the next line.

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	2d6h	v1.34.11
k8s-worker1	Ready	<none>	2d6h	v1.34.11
k8s-worker2	Ready	<none>	2d6h	v1.34.11

Objective: Create an Nginx Deployment with 3 replicas and use the default RollingUpdate strategy.

Step 1: Create YAML file

nano deployment-rolling.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-rolling  
spec:  
  replicas: 3  
  
  strategy:  
    type: RollingUpdate  
    rollingUpdate:  
      maxSurge: 1  
      maxUnavailable: 1  
  
  selector:  
    matchLabels:  
      app: nginx  
  
  template:  
    metadata:  
      labels:  
        app: nginx  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.25  
        ports:  
        - containerPort: 80
```

Step 2: Create Deployment

kubectl apply -f deployment-rolling.yaml

Expected:

deployment.apps/nginx-rolling created

Step 3: Verify Deployment

kubectl get deployments

Expected:

NAME	READY	UP-TO-DATE	AVAILABLE
nginx-rolling	3/3	3	3

Step 4: Check Pods

kubectl get pods -o wide

You should see 3 Nginx pods.

```

ubuntu@k8s-master:~$ kubectl apply -f deployment-rolling.yaml
deployment.apps/nginx-rolling created
ubuntu@k8s-master:~$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
edit-deployment 1/1     1            1           31h
nginx-rolling  3/3     3            3           13s
ubuntu@k8s-master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP
edit-deployment-89c465f4f-qwbtt 1/1     Running   0          8h   192.168.194.92
nginx-rolling-77bf8679f9-9hx4k 1/1     Running   0          27s   192.168.126.32
nginx-rolling-77bf8679f9-hk92n 1/1     Running   0          27s   192.168.194.96
nginx-rolling-77bf8679f9-xmjzz 1/1     Running   0          27s   192.168.126.33
nginx-rs-exclude-backend-2xtb9 1/1     Running   1 (9m35s ago) 8h   192.168.194.94
nginx-rs-exclude-backend-6vqvz 1/1     Running   1 (9m35s ago) 8h   192.168.126.29
nginx-rs-exclude-backend-ffpw5 1/1     Running   1 (9m33s ago) 8h   192.168.126.30
nginx-rs-multiple-labels-bvwmq 1/1     Running   1 (9m33s ago) 8h   192.168.194.95
nginx-rs-multiple-labels-kglc2 1/1     Running   1 (9m35s ago) 8h   192.168.126.31
nginx-rs-multiple-labels-nq7bg 1/1     Running   1 (9m33s ago) 8h   192.168.126.32
ubuntu@k8s-master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE          NOMINATED NODE   READINESS GATES
edit-deployment-89c465f4f-qwbtt 1/1     Running   1 (9m52s ago) 8h   192.168.194.92  k8s-worker1   <none>           <none>
nginx-rolling-77bf8679f9-9hx4k 1/1     Running   0          44s   192.168.126.32  k8s-worker2   <none>           <none>
nginx-rolling-77bf8679f9-hk92n 1/1     Running   0          44s   192.168.194.96  k8s-worker1   <none>           <none>
nginx-rolling-77bf8679f9-xmjzz 1/1     Running   0          44s   192.168.126.33  k8s-worker2   <none>           <none>
nginx-rs-exclude-backend-2xtb9 1/1     Running   1 (9m52s ago) 8h   192.168.194.94  k8s-worker1   <none>           <none>
nginx-rs-exclude-backend-6vqvz 1/1     Running   1 (9m52s ago) 8h   192.168.194.93  k8s-worker1   <none>           <none>
nginx-rs-exclude-backend-ffpw5 1/1     Running   1 (9m50s ago) 8h   192.168.126.29  k8s-worker2   <none>           <none>
nginx-rs-multiple-labels-bvwmq 1/1     Running   1 (9m50s ago) 8h   192.168.126.30  k8s-worker2   <none>           <none>
nginx-rs-multiple-labels-kglc2 1/1     Running   1 (9m52s ago) 8h   192.168.194.95  k8s-worker1   <none>           <none>
nginx-rs-multiple-labels-nq7bg 1/1     Running   1 (9m50s ago) 8h   192.168.126.31  k8s-worker2   <none>           <none>
ubuntu@k8s-master:~$

```

Step 5: Check strategy

kubectl describe deployment nginx-rolling

Look for:

StrategyType: RollingUpdate

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl describe deployment nginx-rolling  
Name: nginx-rolling  
Namespace: default  
CreationTimestamp: Thu, 24 Sep 2026 16:30:16 +0000  
Labels: <none>  
Annotations: deployment.kubernetes.io/revision: 1  
Selector: app=nginx  
Replicas: 3 desired | 3 updated | 3 total | 3 available | 0 unavail  
able  
StrategyType: RollingUpdate  
MinReadySeconds: 0  
RollingUpdateStrategy: 1 max unavailable, 1 max surge  
Pod Template:  
  Labels: app=nginx  
  Containers:  
    nginx:  
      Image: nginx:1.25  
      Port: 80/TCP  
      Host Port: 0/TCP  
      Environment: <none>  
      Mounts: <none>  
      Volumes: <none>  
      Node-Selectors: <none>  
      Tolerations: <none>  
Conditions:  
  Type          Status  Reason  
  ----          -  
  Available     True    MinimumReplicasAvailable  
  Progressing   True    NewReplicaSetAvailable  
OldReplicaSets: <none>  
NewReplicaSet: nginx-rolling-77bf8679f9 (3/3 replicas created)  
Events:  
  Type          Reason          Age    From          Message  
  ----          -  
  Normal        ScalingReplicaSet  2m34s  deployment-controller  Scaled up replica set  
nginx-rolling-77bf8679f9 from 0 to 3  
ubuntu@k8s-master:~$
```

Step 6: Perform a rolling update

Change:

image: nginx:1.25

to:

image: nginx:1.27

Apply:

kubectl apply -f deployment-rolling.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 deployment-rolling.yaml *  
type: RollingUpdate  
rollingUpdate:  
  maxSurge: 1  
  maxUnavailable: 1  
  
selector:  
  matchLabels:  
    app: nginx  
  
template:  
  metadata:  
    labels:  
      app: nginx  
  spec:  
    containers:  
    - name: nginx  
      image: nginx:1.27  
      ports:  
      - containerPort: 80
```

Apply:

kubectl apply -f deployment-rolling.yaml

Step 7: Watch the update

kubectl rollout status deployment/nginx-rolling

You can also watch Pods:

kubectl get pods -w

The old Pods will gradually be replaced by new Pods.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-rolling  
deployment "nginx-rolling" successfully rolled out  
ubuntu@k8s-master:~$ kubectl get pods -w  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-rolling-7489fb554f-5fhsp      1/1     Running   1 (116s ago)  12h  
nginx-rolling-7489fb554f-qpd4m      1/1     Running   1 (116s ago)  12h  
nginx-rolling-7489fb554f-wk7b6      1/1     Running   1 (113s ago)  12h  
s|
```

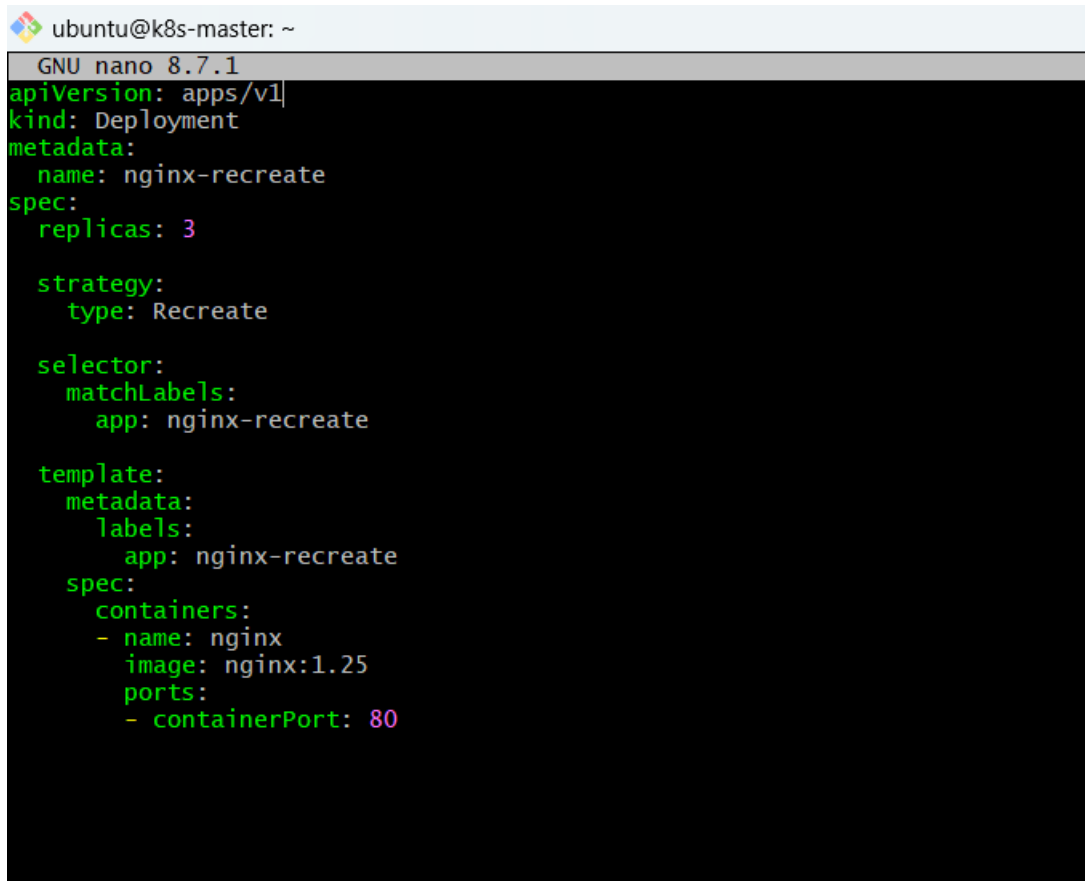
2. Configure a Deployment with a Recreate strategy and observe the downtime.

Objective

Create a Deployment using the **Recreate** strategy and observe that old Pods are deleted before new Pods are created.

Step 1: Create YAML

nano deployment-recreate.yaml

A screenshot of a terminal window on a system named 'ubuntu@k8s-master'. The terminal shows the 'nano' text editor editing a file. The content of the file is a Kubernetes Deployment YAML configuration for 'nginx-recreate'. The configuration specifies a 'Recreate' strategy, 3 replicas, and a container named 'nginx' using the 'nginx:1.25' image on port 80. The terminal text is as follows:

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-recreate  
spec:  
  replicas: 3  
  
  strategy:  
    type: Recreate  
  
  selector:  
    matchLabels:  
      app: nginx-recreate  
  
  template:  
    metadata:  
      labels:  
        app: nginx-recreate  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.25  
        ports:  
        - containerPort: 80
```

Step 2: Apply

kubectl apply -f deployment-recreate.yaml

Step 3: Verify

kubectl get deployment nginx-recreate

kubectl get pods -l app=nginx-recreate

You should have 3 Pods.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano deployment-recreate.yaml  
ubuntu@k8s-master:~$ kubectl apply -f deployment-recreate.yaml  
deployment.apps/nginx-recreate created  
ubuntu@k8s-master:~$ kubectl get deployment nginx-recreate  
NAME          READY   UP-TO-DATE   AVAILABLE   AGE  
nginx-recreate 3/3      3            3           8s  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-recreate  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-recreate-5844f657c7-5424n     1/1     Running   0          19s  
nginx-recreate-5844f657c7-lv9zf     1/1     Running   0          19s  
nginx-recreate-5844f657c7-sq7ng     1/1     Running   0          19s  
ubuntu@k8s-master:~$ |
```

Step 4: Update the image

Edit:

nano deployment-recreate.yaml

Change:

image: nginx:1.25

to:

image: nginx:1.27

Apply:

kubectl apply -f deployment-recreate.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 deployment-recreate.yaml  
replicas: 3  
  
strategy:  
  type: Recreate  
  
selector:  
  matchLabels:  
    app: nginx-recreate  
  
template:  
  metadata:  
    labels:  
      app: nginx-recreate  
  spec:  
    containers:  
    - name: nginx  
      image: nginx:1.27  
      ports:  
      - containerPort: 80
```

Step 5: Watch Pods

Run:

kubectl get pods -l app=nginx-recreate -w

During the update, the old Pods are terminated before the new Pods are created.

This demonstrates the **downtime characteristic of Recreate**.

```
ubuntu@k8s-master: ~  
Volumes: <none>  
Node-Selectors: <none>  
Tolerations: <none>  
Conditions:  
Type Status Reason  
-----  
Available True MinimumReplicasAvailable  
Progressing True NewReplicaSetAvailable  
OldReplicaSets: nginx-recreate-5844f657c7 (0/0 replicas created)  
NewReplicaSet: nginx-recreate-95d84d687 (3/3 replicas created)  
Events:  
Type Reason Age From Message  
-----  
Normal ScalingReplicaSet 6m6s deployment-controller Scaled up replica set nginx-recreate-5844f657c7 from 0 to 3  
Normal ScalingReplicaSet 3m39s deployment-controller Scaled down replica set nginx-recreate-5844f657c7 from 3 to 0  
Normal ScalingReplicaSet 3m37s deployment-controller Scaled up replica set nginx-recreate-95d84d687 from 0 to 3  
  
Name: nginx-rolling  
Namespace: default  
CreationTimestamp: Thu, 24 Sep 2026 16:30:16 +0000  
Labels: <none>  
Annotations: deployment.kubernetes.io/revision: 2  
Selector: app=nginx  
Replicas: 3 desired | 3 updated | 3 total | 3 available | 0 unavailable  
StrategyType: RollingUpdate  
MinReadySeconds: 0  
RollingUpdateStrategy: 1 max unavailable, 1 max surge  
Pod Template:  
Labels: app=nginx  
Containers:  
nginx:  
Image: nginx:1.27  
Port: 80/TCP  
Host Port: 0/TCP  
Environment: <none>  
Mounts: <none>  
Volumes: <none>  
Node-Selectors: <none>  
Tolerations: <none>  
Conditions:  
Type Status Reason  
-----  
Progressing True NewReplicaSetAvailable  
Available True MinimumReplicasAvailable  
OldReplicaSets: nginx-rolling-77bf8679f9 (0/0 replicas created)  
NewReplicaSet: nginx-rolling-7489fb554f (3/3 replicas created)  
Events:  
Type Reason Age From Message  
-----  
Normal ScalingReplicaSet 12h deployment-controller Scaled up replica set nginx-rolling-77bf8679f9 from 0 to 3  
Normal ScalingReplicaSet 12h deployment-controller Scaled up replica set nginx-rolling-7489fb554f from 0 to 1  
Normal ScalingReplicaSet 12h deployment-controller Scaled down replica set nginx-rolling-77bf8679f9 from 3 to 2  
Normal ScalingReplicaSet 12h deployment-controller Scaled up replica set nginx-rolling-7489fb554f from 1 to 2  
Normal ScalingReplicaSet 12h deployment-controller Scaled down replica set nginx-rolling-77bf8679f9 from 2 to 1  
Normal ScalingReplicaSet 12h deployment-controller Scaled up replica set nginx-rolling-7489fb554f from 2 to 3  
Normal ScalingReplicaSet 12h deployment-controller Scaled down replica set nginx-rolling-77bf8679f9 from 1 to 0  
ubuntu@k8s-master:~$
```

3.Update an existing Deployment and perform a rollback to the previous version.

Objective

Deploy version 1.25, update to 1.27, then roll back to 1.25.

Step 1: Create Deployment

nano rollback-deployment.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-rollback  
spec:  
  replicas: 3  
  
  selector:  
    matchLabels:  
      app: nginx-rollback  
  
  template:  
    metadata:  
      labels:  
        app: nginx-rollback  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.25  
        ports:  
        - containerPort: 80
```

Apply:

```
kubectl apply -f rollback-deployment.yaml
```

Step 2: Check rollout history

```
kubectl rollout history deployment/nginx-rollback
```

Step 3: Update to new version

```
kubectl set image deployment/nginx-rollback nginx=nginx:1.27
```

Check:

```
kubectl rollout status deployment/nginx-rollback
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano rollback-deployment.yaml  
ubuntu@k8s-master:~$ kubectl apply -f rollback-deployment.yaml  
deployment.apps/nginx-rollback created  
ubuntu@k8s-master:~$ kubectl rollout history deployment/nginx-rollback  
deployment.apps/nginx-rollback  
REVISION  CHANGE-CAUSE  
1          <none>  
  
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-rollback nginx=nginx:1.27  
deployment.apps/nginx-rollback image updated  
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-rollback  
deployment "nginx-rollback" successfully rolled out  
ubuntu@k8s-master:~$
```

Step 4: Verify image

kubectl get deployment nginx-rollback -o wide

Or:

kubectl describe deployment nginx-rollback

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl get deployment nginx-rollback -o wide
NAME          READY  UP-TO-DATE  AVAILABLE  AGE  CONTAINERS  IMAGES
nginx-rollback 3/3      3            3           2m5s  nginx       nginx:1.27
  app=nginx-rollback
ubuntu@k8s-master:~$ kubectl describe deployment nginx-rollback
Name:          nginx-rollback
Namespace:     default
CreationTimestamp: Fri, 25 Sep 2026 05:02:34 +0000
Labels:        <none>
Annotations:   deployment.kubernetes.io/revision: 2
Selector:      app=nginx-rollback
Replicas:      3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:  RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx-rollback
  Containers:
    nginx:
      Image:      nginx:1.27
      Port:      80/TCP
      Host Port:  0/TCP
      Environment: <none>
      Mounts:      <none>
      Volumes:      <none>
      Node-Selectors: <none>
      Tolerations:  <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  nginx-rollback-6f78c59d9b (0/0 replicas created)
NewReplicaSet:   nginx-rollback-6ff6bd4898 (3/3 replicas created)
Events:
  Type     Reason              Age   From              Message
  ----     -
  Normal   ScalingReplicaSet   2m20s deployment-controller Scaled up replica set nginx-rollback-6f78c59d9b from 0 to 3
  Normal   ScalingReplicaSet   114s  deployment-controller Scaled up replica set nginx-rollback-6ff6bd4898 from 0 to 1
  Normal   ScalingReplicaSet   113s  deployment-controller Scaled down replica set nginx-rollback-6f78c59d9b from 3 to 2
  Normal   ScalingReplicaSet   113s  deployment-controller Scaled up replica set nginx-rollback-6ff6bd4898 from 1 to 2
  Normal   ScalingReplicaSet   112s  deployment-controller Scaled down replica set nginx-rollback-6f78c59d9b from 2 to 1
  Normal   ScalingReplicaSet   112s  deployment-controller Scaled up replica set nginx-rollback-6ff6bd4898 from 2 to 3
  Normal   ScalingReplicaSet   111s  deployment-controller Scaled down replica set nginx-rollback-6f78c59d9b from 1 to 0
ubuntu@k8s-master:~$

```

Step 5: Check rollout history

`kubectl rollout history deployment/nginx-rollback`

You should see multiple revisions.

```

ubuntu@k8s-master:~$ kubectl rollout history deployment/nginx-rollback
deployment.apps/nginx-rollback
REVISION  CHANGE-CAUSE
2         <none>
3         <none>

```

Step 6: Roll back

`kubectl rollout undo deployment/nginx-rollback`

Step 7: Verify rollback

`kubectl rollout status deployment/nginx-rollback`

Then:

kubectl describe deployment nginx-rollback

Check that the image is back to:

nginx:1.25

```
ubuntu@k8s-master: ~  
Labels: <none>  
Annotations: deployment.kubernetes.io/revision: 3  
Selector: app=nginx-rollback  
Replicas: 3 desired | 3 updated | 3 total | 3 available | 0 unavaila  
lable  
StrategyType: RollingUpdate  
MinReadySeconds: 0  
RollingUpdateStrategy: 25% max unavailable, 25% max surge  
Pod Template:  
  Labels: app=nginx-rollback  
  Containers:  
    nginx:  
      Image: nginx:1.25  
      Port: 80/TCP  
      Host Port: 0/TCP  
      Environment: <none>  
      Mounts: <none>  
      Volumes: <none>  
      Node-Selectors: <none>  
      Tolerations: <none>  
Conditions:  
  Type      Status      Reason  
  ----      -  
  Available True        MinimumReplicasAvailable  
  Progressing True        NewReplicaSetAvailable  
OldReplicaSets: nginx-rollback-6ff6bd4898 (0/0 replicas created)  
NewReplicaSet: nginx-rollback-6f78c59d9b (3/3 replicas created)  
Events:  
  Type      Reason      Age      From      Message  
  ----      -  
  Normal    ScalingReplicaSet  6m5s    deployment-controller    Scaled up  
  replica set nginx-rollback-6f78c59d9b from 0 to 3  
  Normal    ScalingReplicaSet  5m39s    deployment-controller    Scaled up  
  replica set nginx-rollback-6ff6bd4898 from 0 to 1  
  Normal    ScalingReplicaSet  5m38s    deployment-controller    Scaled do  
  wn replica set nginx-rollback-6f78c59d9b from 3 to 2  
  Normal    ScalingReplicaSet  5m38s    deployment-controller    Scaled up  
  replica set nginx-rollback-6ff6bd4898 from 1 to 2  
  Normal    ScalingReplicaSet  5m37s    deployment-controller    Scaled do  
  wn replica set nginx-rollback-6f78c59d9b from 2 to 1  
  Normal    ScalingReplicaSet  5m37s    deployment-controller    Scaled up  
  replica set nginx-rollback-6ff6bd4898 from 2 to 3  
  Normal    ScalingReplicaSet  5m36s    deployment-controller    Scaled do  
  wn replica set nginx-rollback-6f78c59d9b from 1 to 0  
  Normal    ScalingReplicaSet  18s     deployment-controller    Scaled up  
  replica set nginx-rollback-6f78c59d9b from 0 to 1  
  Normal    ScalingReplicaSet  17s     deployment-controller    Scaled do  
  wn replica set nginx-rollback-6ff6bd4898 from 3 to 2  
  Normal    ScalingReplicaSet  15s (x4 over 17s) deployment-controller    (combined  
  from similar events): Scaled down replica set nginx-rollback-6ff6bd4898 from 1  
  to 0  
ubuntu@k8s-master:~$ kubectl rollout history deployment/nginx-rollback  
deployment.apps/nginx-rollback  
REVISION  CHANGE-CAUSE  
2          <none>  
3          <none>
```

Useful command

To roll back to a specific revision:

kubectl rollout undo deployment/nginx-rollback --to-revision=1

4.Modify a Deployment to add resource requests and limits for CPU and memory.

Add CPU and Memory Requests and Limits

Objective

Configure resource **requests** and **limits** for the Nginx container.

- Request = minimum resources requested from scheduler
- Limit = maximum resources the container can use

Step 1: Create YAML

nano deployment-resources.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-resources  
spec:  
  replicas: 2  
  
  selector:  
    matchLabels:  
      app: nginx-resources  
  
  template:  
    metadata:  
      labels:  
        app: nginx-resources  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.27  
        ports:  
        - containerPort: 80  
  
        resources:  
          requests:  
            cpu: "100m"  
            memory: "128Mi"  
  
          limits:  
            cpu: "500m"  
            memory: "256Mi"
```

Step 2: Apply

kubectl apply -f deployment-resources.yaml

Step 3: Check Pods

kubectl get pods -l app=nginx-resources

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano deployment-resources.yaml  
ubuntu@k8s-master:~$ kubectl apply -f deployment-resources.yaml  
deployment.apps/nginx-resources created  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-resources  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-resources-974c76b4c-glpb7     1/1     Running   0           14s  
nginx-resources-974c76b4c-z89xj     1/1     Running   0           14s  
ubuntu@k8s-master:~$ |
```

Step 4: Verify resources

`kubectl describe pod <pod-name>`

Look for:

Requests:

cpu: 100m

memory: 128Mi

Limits:

cpu: 500m

memory: 256Mi

You can also check:

```

ubuntu@k8s-master: ~
IP: 192.168.126.60
IPs:
  IP: 192.168.126.60
Controlled By: ReplicaSet/nginx-resources-974c76b4c
Containers:
  nginx:
    Container ID: containerd://f29a6d58bb1a50c0c553c785f38e02ab2e6b5c48008faac29b8ce2308fa738dc
    Image: nginx:1.27
    Image ID: docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port: 80/TCP
    Host Port: 0/TCP
    State: Running
    Started: Fri, 25 Sep 2026 05:16:18 +0000
    Ready: True
    Restart Count: 0
    Limits:
      cpu: 500m
      memory: 256Mi
    Requests:
      cpu: 100m
      memory: 128Mi
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-4958g (
ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers          True
  Initialized                         True
  Ready                              True
  ContainersReady                    True
  PodScheduled                       True
Volumes:
  kube-api-access-4958g:
    Type: Projected (a volume that contains injected data fro
m multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists fo
r 300s
             node.kubernetes.io/unreachable:NoExecute op=Exists
for 300s
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   2m29s default-scheduler Successfully assigned default/ngi
nx-resources-974c76b4c-glpb7 to k8s-worker2
  Normal  Pulled      2m29s kubelet       Container image "nginx:1.27" alre
ady present on machine
  Normal  Created     2m29s kubelet       Created container: nginx
  Normal  Started     2m29s kubelet       Started container nginx
ubuntu@k8s-master:~$

```

You can also check:

`kubectl get deployment nginx-resources -o yaml`

```
ubuntu@k8s-master: ~$ kubectl get deployment nginx-resources -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  labels:
    app: nginx-resources
  name: nginx-resources
  namespace: default
  uid: 7a50377c-7a6f-4ece-ab2b-b9b8bb9ac779
spec:
  progressDeadlineSeconds: 600
  replicas: 2
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: nginx-resources
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
      type: RollingUpdate
  template:
    metadata:
      labels:
        app: nginx-resources
    spec:
      containers:
      - image: nginx:1.27
        imagePullPolicy: IfNotPresent
        name: nginx
        ports:
        - containerPort: 80
          protocol: TCP
        resources:
          limits:
            cpu: 500m
            memory: 256Mi
          requests:
            cpu: 100m
            memory: 128Mi
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
      dnsPolicy: ClusterFirst
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
status:
  availableReplicas: 2
  conditions:
  - lastTransitionTime: "2026-09-25T05:16:19Z"
```

5.Create a Deployment with MaxSurge and MaxUnavailable configurations.

Deployment with MaxSurge and MaxUnavailable

Objective

Control how many Pods can be created or unavailable during a rolling update.

Create:

nano deployment-surge.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-surge  
spec:  
  replicas: 5  
  
  strategy:  
    type: RollingUpdate  
    rollingUpdate:  
      maxSurge: 2  
      maxUnavailable: 1  
  
  selector:  
    matchLabels:  
      app: nginx-surge  
  
  template:  
    metadata:  
      labels:  
        app: nginx-surge  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.25|  
        ports:  
        - containerPort: 80
```

Step 1: Apply

kubectl apply -f deployment-surge.yaml

Step 2: Verify

kubectl get deployment nginx-surge

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f deployment-surge.yaml  
deployment.apps/nginx-surge created  
ubuntu@k8s-master:~$ kubectl get deployment nginx-surge  
NAME          READY   UP-TO-DATE   AVAILABLE   AGE  
nginx-surge   5/5     5            5           9s  
ubuntu@k8s-master:~$
```

Step 3: Update image

kubectl set image deployment/nginx-surge nginx=nginx:1.27

Step 4: Watch rollout

kubectl get pods -l app=nginx-surge -w

Meaning

We configured:

maxSurge: 2

For 5 desired replicas, Kubernetes can temporarily create up to **2 additional Pods** during the update.

So the number of Pods can temporarily reach:

$$5 + 2 = 7$$

And:

maxUnavailable: 1

means Kubernetes allows up to **1 Pod to be unavailable** during the rollout.

Check:

kubectl describe deployment nginx-surge

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-surge nginx=nginx:1.27  
deployment.apps/nginx-surge image updated  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-surge -w  
NAME                                READY    STATUS    RESTARTS   AGE  
nginx-surge-594f995686-dmr7t        1/1      Running   0           45s  
nginx-surge-594f995686-hq7d4        1/1      Running   0           43s  
nginx-surge-594f995686-lj7tc        1/1      Running   0           44s  
nginx-surge-594f995686-qgv2x        1/1      Running   0           45s  
nginx-surge-594f995686-qrs6v        1/1      Running   0           45s  
^Cubuntu@k8s-master:~$ kubectl get pods  
NAME                                READY    STATUS    RESTARTS   AGE  
nginx-recreate-95d84d687-pqx4n      1/1      Running   0           33m  
nginx-recreate-95d84d687-qddfv      1/1      Running   0           33m  
nginx-recreate-95d84d687-srddg      1/1      Running   0           33m  
nginx-resources-974c76b4c-gljb7     1/1      Running   0           11m  
nginx-resources-974c76b4c-z89xj     1/1      Running   0           11m  
nginx-rollback-6f78c59d9b-9lg8n     1/1      Running   0           19m  
nginx-rollback-6f78c59d9b-b6fwg     1/1      Running   0           19m  
nginx-rollback-6f78c59d9b-sg6rm     1/1      Running   0           19m  
nginx-rolling-7489fb554f-5fhsp       1/1      Running   1 (40m ago)  12h  
nginx-rolling-7489fb554f-qpd4m       1/1      Running   1 (40m ago)  12h  
nginx-rolling-7489fb554f-wk7b6      1/1      Running   1 (40m ago)  12h  
nginx-surge-594f995686-dmr7t        1/1      Running   0           2m13s  
nginx-surge-594f995686-hq7d4        1/1      Running   0           2m11s  
nginx-surge-594f995686-lj7tc        1/1      Running   0           2m12s  
nginx-surge-594f995686-qgv2x        1/1      Running   0           2m13s  
nginx-surge-594f995686-qrs6v        1/1      Running   0           2m13s  
ubuntu@k8s-master:~$
```

6.Set up a Deployment with a custom revision history limit.

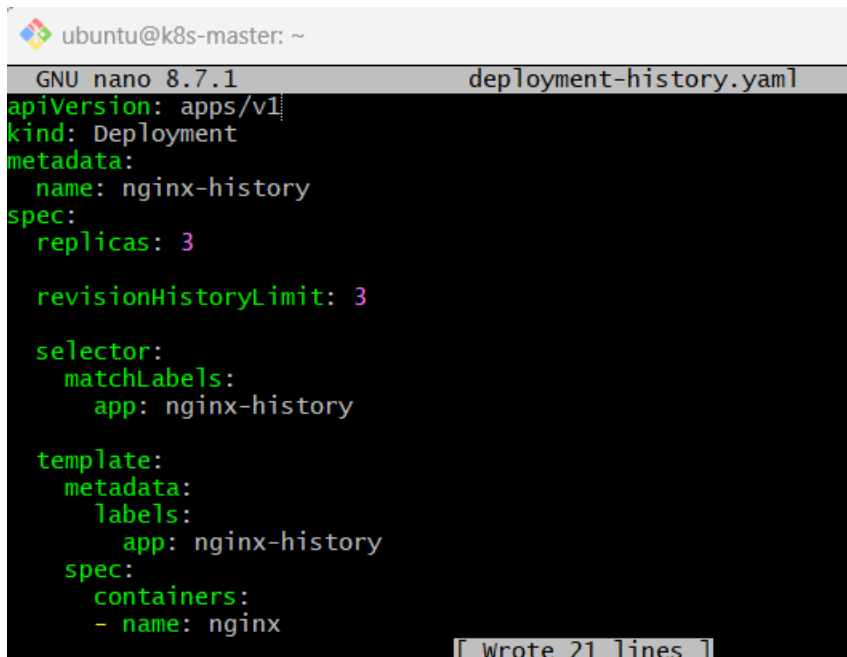
Deployment with Custom Revision History Limit

Objective

Control how many old ReplicaSets Kubernetes keeps for rollback.

Step 1: Create YAML

nano deployment-history.yaml



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 deployment-history.yaml  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-history  
spec:  
  replicas: 3  
  
  revisionHistoryLimit: 3  
  
  selector:  
    matchLabels:  
      app: nginx-history  
  
  template:  
    metadata:  
      labels:  
        app: nginx-history  
    spec:  
      containers:  
      - name: nginx
```

Wrote 21 lines

Step 2: Apply

kubectl apply -f deployment-history.yaml

Step 3: Check Deployment

kubectl get deployment nginx-history

Step 4: Check ReplicaSets

kubectl get replicaset

Or:

kubectl get rs -l app=nginx-history

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f deployment-history.yaml  
deployment.apps/nginx-history created  
ubuntu@k8s-master:~$ kubectl get deployment nginx-history  
NAME          READY   UP-TO-DATE   AVAILABLE   AGE  
nginx-history  3/3     3            3           7s  
ubuntu@k8s-master:~$ kubectl get replicaset  
NAME                                DESIRED   CURRENT   READY   AGE  
nginx-history-6775486b69            3         3         3       22s  
nginx-recreate-5844f657c7           0         0         0       39m  
nginx-recreate-95d84d687            3         3         3       37m  
nginx-resources-974c76b4c           2         2         2       15m  
nginx-rollback-6f78c59d9b           3         3         3       28m  
nginx-rollback-6ff6bd4898           0         0         0       28m  
nginx-rolling-7489fb554f            3         3         3       12h  
nginx-rolling-77bf8679f9            0         0         0       13h  
nginx-surge-594f995686              5         5         5       5m34s  
nginx-surge-fbcb45df6               0         0         0       7m7s  
ubuntu@k8s-master:~$ kubectl get rs -l app=nginx-history  
NAME                                DESIRED   CURRENT   READY   AGE  
nginx-history-6775486b69            3         3         3       43s  
ubuntu@k8s-master:~$ |
```

Step 5: Perform updates

For example:

```
kubectl set image deployment/nginx-history nginx=nginx:1.26
```

Then:

```
kubectl set image deployment/nginx-history nginx=nginx:1.27
```

Then:

```
kubectl set image deployment/nginx-history nginx=nginx:1.28
```

Check:

```
kubectl get rs -l app=nginx-history
```

The Deployment is configured to retain up to **3 old revisions** for rollback, subject to Kubernetes' ReplicaSet cleanup behavior.

```
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-history nginx=nginx:1.26
deployment.apps/nginx-history image updated
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-history nginx=nginx:1.27
deployment.apps/nginx-history image updated
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-history nginx=nginx:1.28
deployment.apps/nginx-history image updated
ubuntu@k8s-master:~$ kubectl get rs -l app=nginx-history
NAME                                DESIRED    CURRENT    READY    AGE
nginx-history-6775486b69            0          0          0        2m24s
nginx-history-776b4b44ff            0          0          0        25s
nginx-history-78f784d558            0          0          0        32s
nginx-history-84b5b6fd88            3          3          3        15s
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-history nginx=nginx:1.27
K
ubuntu@k8s-master:~$ kubectl describe deployment/nginx-history
Name:                               nginx-history
Namespace:                         default
CreationTimestamp:                  Fri, 25 Sep 2026 05:31:01 +0000
Labels:                             <none>
Annotations:                        deployment.kubernetes.io/revision: 4
Selector:                           app=nginx-history
Replicas:                           3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:                       RollingUpdate
MinReadySeconds:                    0
RollingUpdateStrategy:              25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx-history
  Containers:
    nginx:
      Image:          nginx:1.28
      Port:           <none>
      Host Port:      <none>
      Environment:    <none>
      Mounts:         <none>
      Volumes:        <none>
      Node-Selectors:  <none>
      Tolerations:    <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  nginx-history-6775486b69 (0/0 replicas created), nginx-history-78f784d558 (0/0 replicas created), nginx-history-776b4b44ff (0/0 replicas created)
NewReplicaSet:   nginx-history-84b5b6fd88 (3/3 replicas created)
Events:
  Type    Reason          Age    From          Message
  ----    -
  Normal  ScalingReplicaSet  3m28s  deployment-controller  Scaled up replica set nginx-history-6775486b69 from 0 to 3
  Normal  ScalingReplicaSet  96s    deployment-controller  Scaled up replica set nginx-history-78f784d558 from 0 to 1
  Normal  ScalingReplicaSet  93s    deployment-controller  Scaled down replica set nginx-history-6775486b69 from 3 to 2
  Normal  ScalingReplicaSet  91s    deployment-controller  Scaled up replica set nginx-history-78f784d558 from 1 to 2
  Normal  ScalingReplicaSet  89s    deployment-controller  Scaled down replica set nginx-history-6775486b69 from 2 to 1
  Normal  ScalingReplicaSet  89s    deployment-controller  Scaled up replica set nginx-history-78f784d558 from 2 to 3
  Normal  ScalingReplicaSet  89s    deployment-controller  Scaled down replica set nginx-history-78f784d558 from 3 to 2
  Normal  ScalingReplicaSet  89s    deployment-controller  Scaled up replica set nginx-history-776b4b44ff from 0 to 1
  Normal  ScalingReplicaSet  88s    deployment-controller  Scaled down replica set nginx-history-6775486b69 from 1 to 0
  Normal  ScalingReplicaSet  71s (x10 over 88s)  deployment-controller  (combined from similar events): Scaled down replica set nginx-history-776b4b44ff from 1 to 0
ubuntu@k8s-master:~$
```

7. Pause a Deployment during an update, and then resume it.

Pause Deployment During an Update and Resume

Objective

Pause a Deployment, make changes while it is paused, then resume it.

Step 1: Create Deployment

nano deployment-pause.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-pause  
spec:  
  replicas: 3  
  
  selector:  
    matchLabels:  
      app: nginx-pause  
  
  template:  
    metadata:  
      labels:  
        app: nginx-pause  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.25
```

Apply:

kubectl apply -f deployment-pause.yaml

Step 2: Verify

kubectl get pods -l app=nginx-pause

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano deployment-pause.yaml  
ubuntu@k8s-master:~$ kubectl apply -f deployment-pause.yaml  
deployment.apps/nginx-pause created  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-pause  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-pause-6c988744dc-shb4s        0/1     Pending   0           7s  
nginx-pause-6c988744dc-zjmqz        0/1     Pending   0           7s  
nginx-pause-6c988744dc-zs6z8        0/1     Pending   0           7s  
ubuntu@k8s-master:~$  
ubuntu@k8s-master:~$ |
```

Step 3: Pause Deployment

kubectl rollout pause deployment/nginx-pause

Verify:

kubectl rollout status deployment/nginx-pause

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl rollout pause deployment/nginx-pause  
deployment.apps/nginx-pause paused  
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-pause  
deployment "nginx-pause" successfully rolled out  
ubuntu@k8s-master:~$
```

Step 4: Change image

`kubectl set image deployment/nginx-pause nginx=nginx:1.26`

Because the Deployment is paused, the rollout does not proceed normally.

You can make another change:

`kubectl set image deployment/nginx-pause nginx=nginx:1.27`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl rollout pause deployment/nginx-pause  
deployment.apps/nginx-pause paused  
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-pause  
deployment "nginx-pause" successfully rolled out  
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-pause nginx=nginx:1.26  
deployment.apps/nginx-pause image updated  
ubuntu@k8s-master:~$ kubectl set image deployment/nginx-pause nginx=nginx:1.27  
deployment.apps/nginx-pause image updated  
ubuntu@k8s-master:~$
```

Step 5: Check

`kubectl get deployment nginx-pause`

Step 6: Resume Deployment

`kubectl rollout resume deployment/nginx-pause`

Step 7: Check rollout

`kubectl rollout status deployment/nginx-pause`

Verify:

`kubectl get pods -l app=nginx-pause`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get deployment nginx-pause  
NAME          READY   UP-TO-DATE   AVAILABLE   AGE  
nginx-pause   3/3     0            3           3m8s  
ubuntu@k8s-master:~$ kubectl rollout resume deployment/nginx-pause  
deployment.apps/nginx-pause resumed  
ubuntu@k8s-master:~$ kubectl rollout status deployment/nginx-pause  
deployment "nginx-pause" successfully rolled out  
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx-pause  
NAME                                READY   STATUS    RESTARTS   AGE  
nginx-pause-f68cc7889-b68kw         1/1     Running   0          26s  
nginx-pause-f68cc7889-fz7k5         1/1     Running   0          29s  
nginx-pause-f68cc7889-jw5nh         1/1     Running   0          28s  
ubuntu@k8s-master:~$ |
```

8. Create a pod using resource requests for memory and CPU, and observe how the scheduler assigns it to a node.

Pod with CPU and Memory Requests — Observe Scheduler

Objective

Create a Pod with CPU and memory **requests** and observe which node Kubernetes Scheduler selects.

The scheduler uses resource requests when determining whether a node has enough allocatable resources.

Step 1: Check worker nodes

`kubectl get nodes -o wide`

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes -o wide  
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION  
k8s-master    Ready     control-plane  2d19h  v1.34.11  172.31.17.127 <none>        Ubuntu 26.04.1 LTS  7.0.0-1012-  
aws           containerd://2.2.2  
k8s-worker1   Ready     <none>     2d19h  v1.34.11  172.31.16.155 <none>        Ubuntu 26.04 LTS    7.0.0-1012-  
aws           containerd://2.2.2  
k8s-worker2   Ready     <none>     2d19h  v1.34.11  172.31.26.51 <none>        Ubuntu 26.04 LTS    7.0.0-1012-  
aws           containerd://2.2.2  
ubuntu@k8s-master:~$ |
```

Step 2: Check node resources

kubectl describe nodes

Look for:

Capacity:

Allocatable:

You can also run:

kubectl top nodes

If Metrics Server is installed, you'll see CPU and memory usage

```
ubuntu@k8s-master: ~  
-----  
NetworkUnavailable      False      Fri, 25 Sep 2026 04:48:09 +0000      Fri, 25 Sep 2026 04:48:09 +0000      CalicoIsUp  
Calico is running on this node  
MemoryPressure          False      Fri, 25 Sep 2026 05:38:29 +0000      Tue, 22 Sep 2026 10:19:34 +0000      KubeletHasSufficientM  
emory kubelet has sufficient memory available  
DiskPressure            False      Fri, 25 Sep 2026 05:38:29 +0000      Fri, 25 Sep 2026 05:38:29 +0000      KubeletHasNoDiskPress  
ure kubelet has no disk pressure  
PIDPressure             False      Fri, 25 Sep 2026 05:38:29 +0000      Tue, 22 Sep 2026 10:19:34 +0000      KubeletHasSufficientP  
ID kubelet has sufficient PID available  
Ready                   True       Fri, 25 Sep 2026 05:38:29 +0000      Tue, 22 Sep 2026 10:20:14 +0000      KubeletReady  
kubelet is posting ready status  
Addresses:  
InternalIP: 172.31.26.51  
Hostname: k8s-worker2  
Capacity:  
cpu: 2  
ephemeral-storage: 6931592Ki  
hugepages-1Gi: 0  
hugepages-2Mi: 0  
memory: 3902880Ki  
pods: 110  
Allocatable:  
cpu: 2  
ephemeral-storage: 6388155177  
hugepages-1Gi: 0  
hugepages-2Mi: 0  
memory: 3800480Ki  
pods: 110  
System Info:  
Machine ID: ec259f2e0997f5eed4d5838492e422b1  
System UUID: ec259f2e-0997-f5ee-d4d5-838492e422b1  
Boot ID: c9bb37e8-6655-4fde-9f86-61fc790f27c8  
Kernel Version: 7.0.0-1012-aws  
OS Image: Ubuntu 26.04 LTS  
Operating System: linux  
Architecture: amd64  
Container Runtime Version: containerd://2.2.2  
Kubelet Version: v1.34.11  
Kube-Proxy Version:  
PodCIDR: 192.168.2.0/24  
PodCIDRs: 192.168.2.0/24  
Non-terminated Pods: 14 in total  
Namespace  
s Age  
-----  
-----  
CPU Requests CPU Limits Memory Requests Memory Limit  
-----  
-----  
default nginx-history-84b56bfd88-4vhv2 0 (0%) 0 (0%) 0 (0%) 0 (0%)  
10m default nginx-history-84b56bfd88-ztf97 0 (0%) 0 (0%) 0 (0%) 0 (0%)  
10m default nginx-pause-f68cc7889-b68kw 0 (0%) 0 (0%) 0 (0%) 0 (0%)  
2m57s default nginx-pause-f68cc7889-fz7k5 0 (0%) 0 (0%) 0 (0%) 0 (0%)  
3m default nginx-recreate-95d84d687-pqx4n 0 (0%) 0 (0%) 0 (0%) 0 (0%)  
49m default nginx-recreate-95d84d687-srddg 0 (0%) 0 (0%) 0 (0%) 0 (0%)  
default
```

```

ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ kubectl top nodes
NAME          CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
k8s-master    88m          4%       1899Mi          51%
k8s-worker1   38m          1%       1113Mi          30%
k8s-worker2   33m          1%       1026Mi          27%
ubuntu@k8s-master:~$ kubectl top pods
NAME          CPU(cores)   MEMORY(bytes)
nginx-history-84b56bfd88-4vhv2    0m          3Mi
nginx-history-84b56bfd88-sdg4g    0m          3Mi
nginx-history-84b56bfd88-ztf97    0m          3Mi
nginx-pause-f68cc7889-b68kw       0m          2Mi
nginx-pause-f68cc7889-fz7k5       0m          2Mi
nginx-pause-f68cc7889-jw5nh       0m          2Mi
nginx-recreate-95d84d687-pqx4n    0m          2Mi
nginx-recreate-95d84d687-qddfv    0m          2Mi
nginx-recreate-95d84d687-srddg    0m          2Mi
nginx-resources-974c76b4c-glpb7    0m          2Mi
nginx-resources-974c76b4c-z89xj    0m          2Mi
nginx-rollback-6f78c59d9b-9lg8n    0m          2Mi
nginx-rollback-6f78c59d9b-b6fwg    0m          2Mi
nginx-rollback-6f78c59d9b-sg6rm    0m          2Mi
nginx-rolling-7489fb554f-5fhsp     0m          7Mi
nginx-rolling-7489fb554f-qpd4m     0m          8Mi
nginx-rolling-7489fb554f-wk7b6     0m          12Mi
nginx-surge-594f995686-dmr7t      0m          2Mi
nginx-surge-594f995686-hq7d4      0m          2Mi
nginx-surge-594f995686-lj7tc      0m          2Mi
nginx-surge-594f995686-qgv2x      0m          2Mi
nginx-surge-594f995686-qrs6v      0m          2Mi
ubuntu@k8s-master:~$ |

```

Step 3: Create Pod YAML

nano resource-pod.yaml

```

ubuntu@k8s-master: ~
GNU nano 8.7.1
apiVersion: v1
kind: Pod
metadata:
  name: scheduler-resource-pod
spec:
  containers:
  - name: nginx
    image: nginx:1.27

    resources:
      requests:
        cpu: "200m"
        memory: "256Mi"

      limits:
        cpu: "500m"
        memory: "512Mi"

```

Step 4: Create Pod

```
kubectl apply -f resource-pod.yaml
```

Expected:

```
pod/scheduler-resource-pod created
```

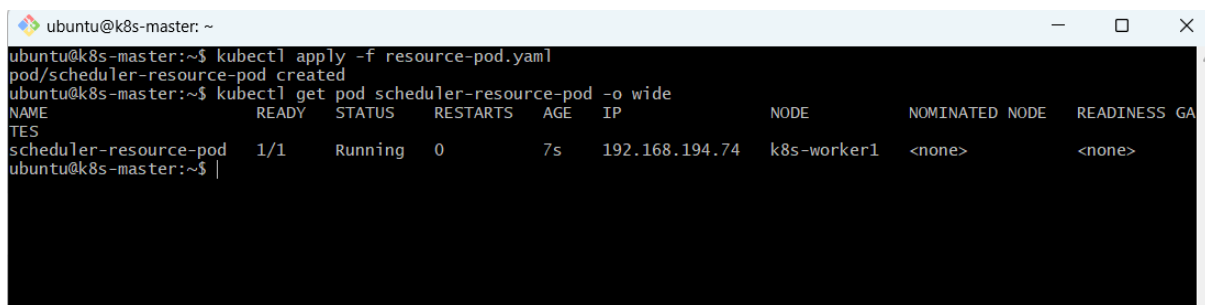
Step 5: Check Pod

```
kubectl get pod scheduler-resource-pod -o wide
```

Example:

NAME	READY	STATUS	IP	NODE
scheduler-resource-pod	1/1	Running	10.244.x.x	k8s-worker1

The NODE column tells you where the scheduler placed the Pod.



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f resource-pod.yaml  
pod/scheduler-resource-pod created  
ubuntu@k8s-master:~$ kubectl get pod scheduler-resource-pod -o wide  
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES  
scheduler-resource-pod 1/1     Running   0          7s    192.168.194.74 k8s-worker1 <none>          <none>
```

Step 6: Check scheduling events

Run:

```
kubectl describe pod scheduler-resource-pod
```

At the bottom, look at:

Events:

You should see something similar to:

Successfully assigned default/scheduler-resource-pod to k8s-worker1

This demonstrates the scheduler's placement decision.

```

ubuntu@k8s-master: ~
Priority: 0
Service Account: default
Node: k8s-worker1/172.31.16.155
Start Time: Fri, 25 Sep 2026 05:56:25 +0000
Labels: <none>
Annotations: cni.projectcalico.org/containerID: 53ae95e6b5d4f34abcc84c1be95291d75c9288ea79002b64934622cf7f00ea80
              cni.projectcalico.org/podIP: 192.168.194.74/32
              cni.projectcalico.org/podIPs: 192.168.194.74/32
Status: Running
IP: 192.168.194.74
IPs: 192.168.194.74
Containers:
  nginx:
    Container ID: containerd://ea9e3065bf73c53bfb3a881021eed0accf41b43562db782f86866521adbbc13
    Image: nginx:1.27
    Image ID: docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Fri, 25 Sep 2026 05:56:26 +0000
    Ready: True
    Restart Count: 0
    Limits:
      cpu: 500m
      memory: 512Mi
    Requests:
      cpu: 200m
      memory: 256Mi
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-nz7tb (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-nz7tb:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   63s   default-scheduler   Successfully assigned default/scheduler-resource-pod to k8s-worker1
  Normal  Pulled      62s   kubelet         Container image "nginx:1.27" already present on machine
  Normal  Created     62s   kubelet         Created container: nginx
  Normal  Started     62s   kubelet         Started container nginx
ubuntu@k8s-master:~$

```

Step 7: Verify requests

Run:

kubectl describe pod scheduler-resource-pod

Find:

Requests:

cpu: 200m

memory: 256Mi

Limits:

cpu: 500m

memory: 512Mi

```

ubuntu@k8s-master: ~
Priority: 0
Service Account: default
Node: k8s-worker1/172.31.16.155
Start Time: Fri, 25 Sep 2026 05:56:25 +0000
Labels: <none>
Annotations: cnj.projectcalico.org/containerID: 53ae95e6b5d4f34abcc84c1be95291d75c9288ea79002b64934622cf7f00ea80
              cnj.projectcalico.org/podIP: 192.168.194.74/32
              cnj.projectcalico.org/podIPs: 192.168.194.74/32
Status: Running
IP: 192.168.194.74
IPs: 192.168.194.74
Containers:
  nginx:
    Container ID: containerd://ea9e3065bf73c53bfb3a881021eed0accf41b43562db782f86866521adbbc13
    Image: nginx:1.27
    Image ID: docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Fri, 25 Sep 2026 05:56:26 +0000
    Ready: True
    Restart Count: 0
    Limits:
      cpu: 500m
      memory: 512Mi
    Requests:
      cpu: 200m
      memory: 256Mi
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-nz7tb (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers          True
  Initialized                         True
  Ready                              True
  ContainersReady                    True
  PodScheduled                       True
Volumes:
  kube-api-access-nz7tb:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type      Reason      Age   From          Message
  ----      -
  Normal    Scheduled   2m10s default-scheduler Successfully assigned default/scheduler-resource-pod to k8s-worker1
  Normal    Pulled      2m9s  kubelet       Container image "nginx:1.27" already present on machine
  Normal    Created     2m9s  kubelet       Created container: nginx
  Normal    Started     2m9s  kubelet       Started container nginx
ubuntu@k8s-master:~$

```

Important Resource Concepts for Your Recording

CPU

1000m = 1 CPU

500m = 0.5 CPU

200m = 0.2 CPU

100m = 0.1 CPU

Memory

128Mi = 128 MiB

256Mi = 256 MiB

512Mi = 512 MiB

1Gi = 1 GiB

Request vs Limit

Configuration	Meaning
requests.cpu	CPU guaranteed/requested for scheduling
requests.memory	Memory requested for scheduling
limits.cpu	Maximum CPU usage allowed
limits.memory	Maximum memory usage allowed

Useful Verification Commands

After completing the labs, these commands are useful for your recording:

Deployments

```
kubectl get deployments
```

```
kubectl get deployment
```

```
kubectl describe deployment <deployment-name>
```

Pods

```
kubectl get pods -o wide
```

```
kubectl describe pod <pod-name>
```

ReplicaSets

```
kubectl get rs
```

Rollout

```
kubectl rollout status deployment/<deployment-name>
```

```
kubectl rollout history deployment/<deployment-name>
```

```
kubectl rollout pause deployment/<deployment-name>
```

```
kubectl rollout resume deployment/<deployment-name>
```

```
kubectl rollout undo deployment/<deployment-name>
```

Check a Deployment's YAML

```
kubectl get deployment <deployment-name> -o yaml
```

Check where Pods are running

kubectl get pods -o wide

Check node resources

kubectl describe node <node-name>

Lab Summary

No. Task	Main Kubernetes Concept
1 3 replicas + rolling update	RollingUpdate
2 Recreate deployment	Recreate
3 Update + rollback	rollout undo
4 CPU/memory resources	requests / limits
5 Surge/unavailable	maxSurge / maxUnavailable
6 Revision history	revisionHistoryLimit
7 Pause/resume	rollout pause/resume
8 Scheduler resources	Resource requests + scheduling